Lab Report: Docker Containers Fundamentals

Student Name: Lu Weichi
Course: Cloud Computing
Date:2026-4-28
Tools Used: Docker Desktop , Git, Notepad, Browser

Table of Contents

Experiment 2.1 – Publishing and Exposing Ports

Experiment 2.2 – Overriding Container Defaults

Experiment 2.3 – Persisting Container Data

Experiment 2.4 – Sharing Local Files with Containers (Bind Mounts)

Experiment 2.5 – Multi- container Applications (Manual vs Docker Compose)

Experiment 2.1 – Publishing and Exposing Ports

Objective:

Learn how to map a container's internal port to a host port so that a web application can be accessed from a browser.

Experimental procedure:

1.Pull and run the welcome-to-docker container with port mapping

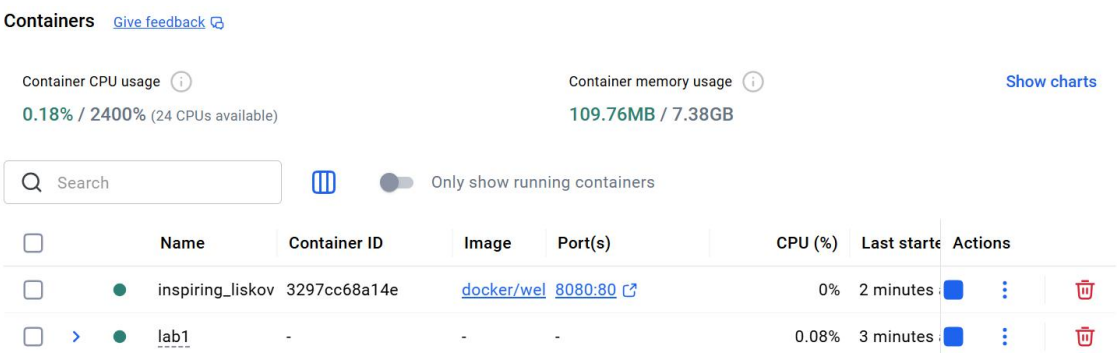
Command:*docker run -d -p 8080:80 docker/welcome-to-docker*

First run downloads the image automatically.

```
PS C:\Users\21315> docker run -d -p 8080:80 docker/welcome-to-docker
Unable to find image 'docker/welcome-to-docker:latest' locally
latest: Pulling from docker/welcome-to-docker
9745203f5d34: Pull complete
fd372c3c84a2: Pull complete
828fa206d77b: Pull complete
bdaad27fd04a: Pull complete
c1d2dc189e38: Pull complete
958a74d6a238: Pull complete
9824c27679d3: Pull complete
a5585638209e: Pull complete
1babe895ddff: Download complete
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest
3297cc68a14e52cc4ff7fb6ca2c35ffc33fa9ffcc4db53cc15708d799402ca6d
PS C:\Users\21315>
```

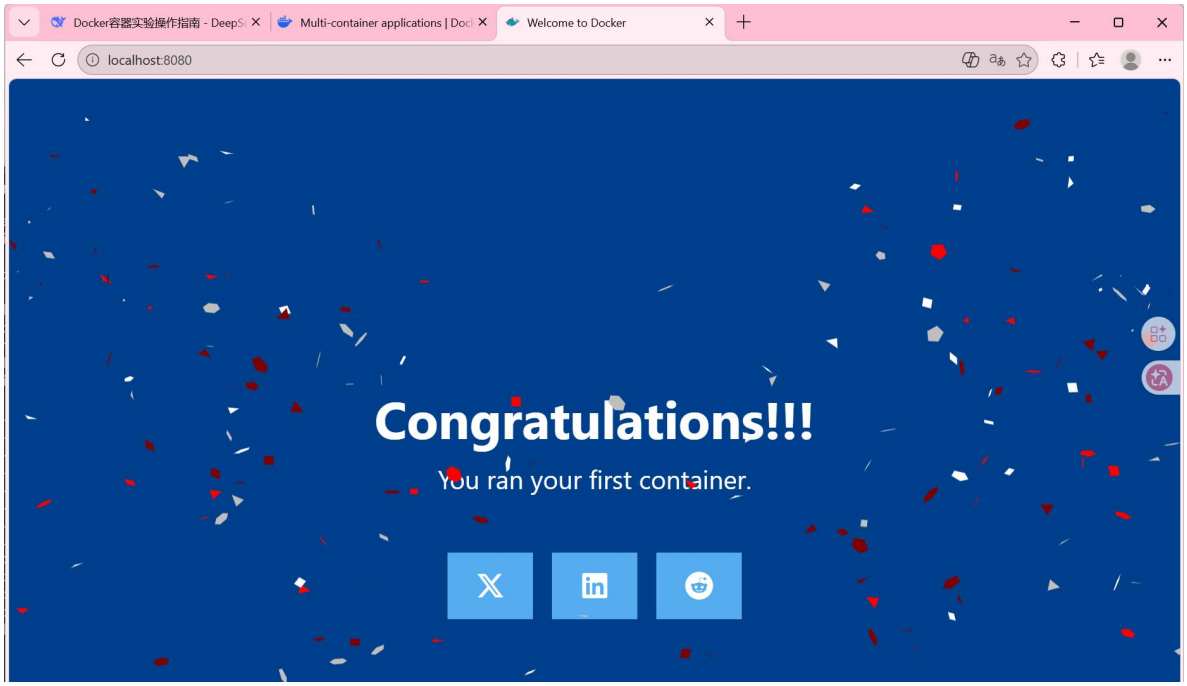
2.Verify the container is running

docker ps



2. Open the web page

Open a browser and go to http://localhost:8080.



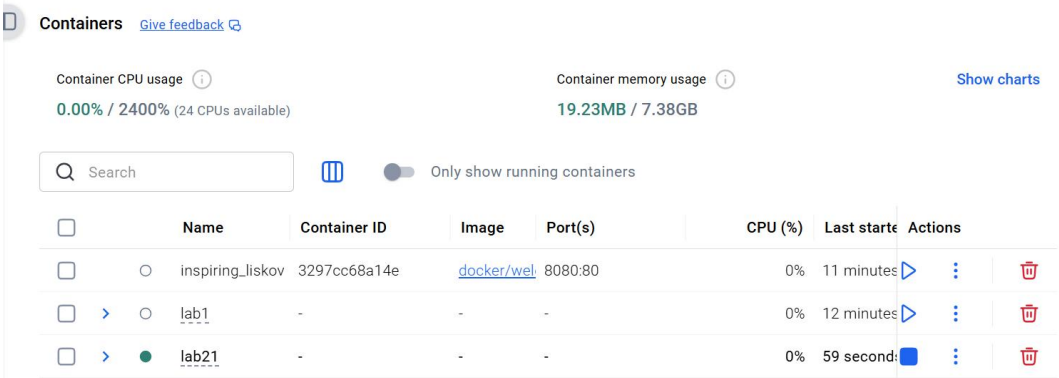
Docker Compose alternative

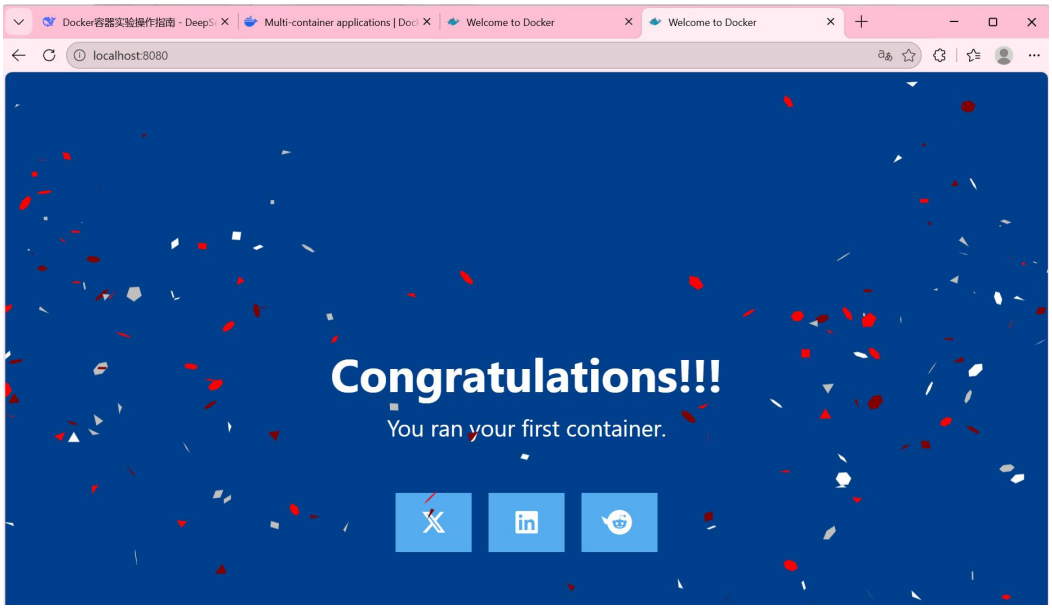
1.Create a compose.yml file in a new folder, create:

2.Start using Compose

docker compose up -d

3.Verify again – same browser page as above.





Experiment 2.2 – Overriding Container Defaults

Objective:

Run multiple PostgreSQL instances on different host ports, attach containers to a custom network, limit resources, and override CMD/ENTRYPOINT with Docker Compose.

Experimental procedure:

(1)Running two Postgres containers on different host ports

1.First Postgres container (host port 5432)

docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres

```
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
e57497b6cf23: Pull complete
13f7f91984c8: Pull complete
e9cad8a9e695: Pull complete
6fadfe1078a3: Pull complete
56b9e522a2ac: Pull complete
85f6ddcd85b0: Pull complete
c43308a19dbe: Pull complete
ce10992ea794: Pull complete
b7036600a44b: Pull complete
2428a49310a9: Pull complete
c3f159f3b349: Pull complete
3531af2bc2a9: Pull complete
4e2103318c3c: Pull complete
6e695e05adc6: Download complete
631e25198ba2: Download complete
Digest: sha256:78481659c47e862334611ccdaf7c369c986b3046da9857112f3b309114a65fb4
Status: Downloaded newer image for postgres:latest
19e3c362b6595b3cdc811dded48439d26fc420343c52ba0c0918a6033c2bd563
```

2.Second Postgres container (host port 5433)

docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres

```
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
7993f276ae6a7abf47e7f4ed0c6883b6c8f513d39ed30dce110a86772853ba66
PS C:\Users\21315>
```

3.Verify both containers :*docker ps*

```
PS C:\Users\21315> docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED              STATUS              PORTS
                                                                  NAMES
7993f276ae6a   postgres            "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0.0.0:5433->5432
/tcp, [::]:5433->5432/tcp                                         upbeat_banach
19e3c362b659   postgres            "docker-entrypoint.s…"   31 minutes ago       Up 31 minutes       0.0.0.0:5432->5432
/tcp, [::]:5432->5432/tcp                                         busy_vaughan
f22879f67d58   gitea/gitea:latest   "/usr/bin/entrypoint…"   25 hours ago         Up 33 minutes       0.0.0.0:3000->3000
/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp   gitea
PS C:\Users\21315>
```

Errors encountered

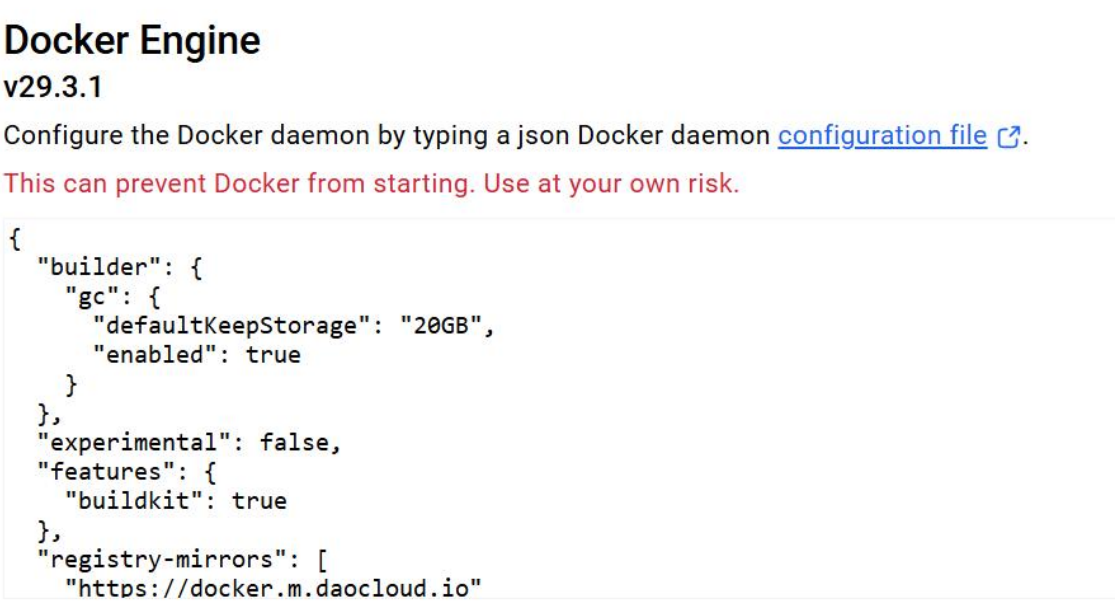
Initial docker pull postgres failed with:

unable to fetch descriptor ... content size of zero: invalid argument

```
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
docker: Error response from daemon: unable to fetch descriptor (sha256:78481659c47e862334611ccdaf7c369c986b3046da9857112
f3b309114a65fb4) which reports content size of zero: invalid argument

Run 'docker run --help' for more information
PS C:\Users\21315>  docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
docker: Error response from daemon: unable to fetch descriptor (sha256:78481659c47e862334611ccdaf7c369c986b3046da9857112
f3b309114a65fb4) which reports content size of zero: invalid argument

Run 'docker run --help' for more information
PS C:\Users\21315> docker pull postgres:15
Error response from daemon: unable to fetch descriptor (sha256:29342cb52157b098821961d2c14eec3c019071f56a5d559e990cf07cf
541ea9b) which reports content size of zero: invalid argument
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
docker: Error response from daemon: unknown: failed to resolve reference "docker.io/library/postgres:latest": unexpected
 status from HEAD request to https://ycndbz5z.mirror.aliyuncs.com/v2/library/postgres/manifests/latest?ns=docker.io: 403
 Forbidden
```

Cause: Failure to fetch the image descriptor due to an unreliable or blocked mirror (https://docker.xuanvuan.me) in the campus network environment, resulting in a zero-byte content response.

Solution: Switched to a stable domestic mirror source (the official Alibaba Cloud Container Registry mirror) and removed the faulty third-party mirror from the Docker daemon configuration, which restored normal image-layer downloads.



Custom network and third Postgres instance

1.Create custom network: *docker network create mynetwork*

```
PS C:\Users\21315> docker network create mynetwork
ace33bd4227458a3404d6dcc9ca74d36d078e071fced297dc0815f7138bf351d
PS C:\Users\21315>
```

2.Run third Postgres attached to mynetwork (host port 5434)

docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres

```
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres
a199ee1258926a6f63eb56eea94e37bedec6e504fc4d34dadac0c5dfd90e879f
PS C:\Users\21315>
```

3.Inspect network: docker *network inspect mynetwork*

```
PS C:\Users\21315> docker network inspect mynetwork
[
    {
        "Name": "mynetwork",
        "Id": "ace33bd4227458a3404d6dcc9ca74d36d078e071fced297dc0815f7138bf351d",
        "Created": "2026-04-27T07:54:37.554195584Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "a199ee1258926a6f63eb56eea94e37bedec6e504fc4d34dadac0c5dfd90e879f": {
                "Name": "distracted_darwin",
                "EndpointID": "aedd2e10bf0966583c5d0bdde1b780c59fff8b2b7fce89082f65082bb4ef0972",
                "MacAddress": "32:9e:d1:31:f8:79",
                "IPv4Address": "172.20.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.20.0.0/16": {
                        "IPsInUse": 4,
                        "DynamicIPsAvailable": 65532
                    }
                }
            }
        }
```

Resource limits

docker run -d -e POSTGRES_PASSWORD=secret --memory="512m" --cpus=".5" postgres

Then run docker stats --no-stream to show memory limit.

	Name	Container ID	Image	Port(s)	CPU (%)	Last starte	Actions
	inspiring_liskov	3297cc68a14e	docker/wel	8080:80	0%	1 hour ago	
	busy_vaughan	19e3c362b659	postgres	5432:5432	0%	34 minutes	
	upbeat_banach	7993f276ae6a	postgres	5433:5432	0%	3 minutes	
	distracted_darw	a199ee125892	postgres	5434:5432	0.07%	1 minute a	
	lab1	-	-	-	0.11%	36 minutes	
	lab21	-	-	-	0%	1 hour ago	

```
PS C:\Users\21315> docker run -d -e POSTGRES_PASSWORD=secret --memory="512m" --cpus=".5" postgres
b9940fc29fa32f91bdb1d2804e3dfc36f059fe6297dbccefac2712cb61e1f5f2
PS C:\Users\21315> docker stats --no-stream
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O
        PIDS
b9940fc29fa3   goofy_almeida       0.06%     33.59MiB / 512MiB     6.56%     872B / 126B     0B / 40.9M
B       9
a199ee125892   distracted_darwin   0.00%     37.35MiB / 7.555GiB   0.48%     1.7kB / 126B    0B / 42.6M
B       9
7993f276ae6a   upbeat_banach       0.00%     37.57MiB / 7.555GiB   0.49%     998B / 126B     0B / 42.3M
B       9
19e3c362b659   busy_vaughan        0.75%     37.96MiB / 7.555GiB   0.49%     1.95kB / 126B   0B / 42.5M
B       9
f22879f67d58   gitea               0.08%     88.84MiB / 7.555GiB   1.15%     1.83kB / 126B   96.7MB / 2
4.6kB   25
PS C:\Users\21315>
```

Overriding CMD/ENTRYPOINT with Docker Compose

1.Create a new folder override-demo and compose.yml

docker-compose.yml | Lab.2.txt | compose.yml

文件 编辑 查看

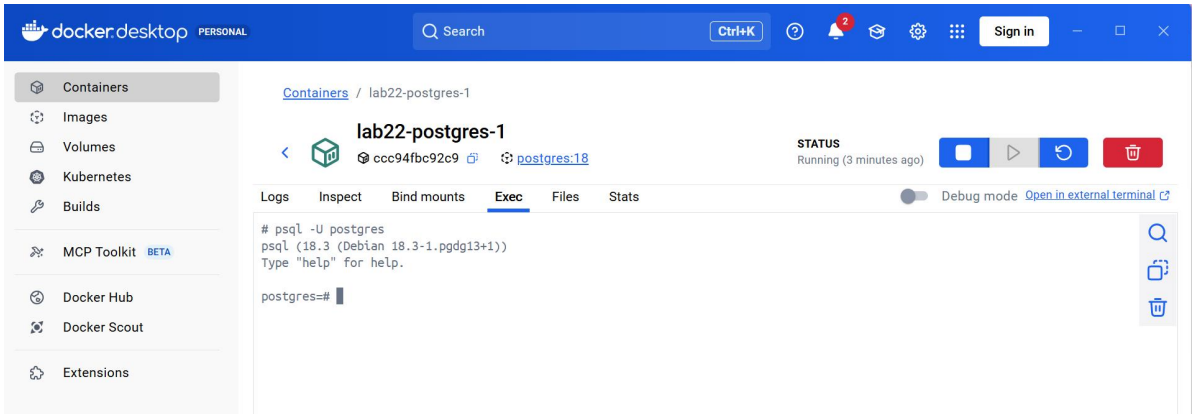
```
services:
  postgres:
    image: postgres:18
    entrypoint: ["docker-entrypoint.sh", "postgres"]
    command: ["-h", "localhost", "-p", "5432"]
    environment:
      POSTGRES_PASSWORD: secret
```

2.Start the service: *docker compose up -d*

```
PS E:\云计算导论\Lab2\Lab2.2> docker compose up -d
[+] up 3/3
 ✔Image postgres:18          Pulled                                    2.2s
 ✔Network lab22_default      Created                                   0.0s
 ✔Container lab22-postgres-1 Started                                   0.4s
PS E:\云计算导论\Lab2\Lab2.2>
```

3.Access the container and run psql

Through Docker Desktop: find the container → Exec tab → type psql -U postgres.



Experiment 2.3 – Persisting Container Data (Volumes)

Objective:

Use Docker volumes to keep database data alive even after the container is deleted.

Experimental procedure:

1.Run Postgres with a named volume

docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql postgres:18

```
PS C:\Users\21315> docker run --name=db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql postgres:18
6c2aa2b6ee294a0afb2373ca40c87b481c5ce622601dd2311414fca015d3dfcb
PS C:\Users\21315>
```

2.Create a table and insert data

docker exec -ti db psql -U postgres

Inside psql:

sql

*CREATE TABLE tasks (id SERIAL PRIMARY KEY, description VARCHAR(100));INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');SELECT * FROM tasks;\q*

```
PS C:\Users\21315> docker exec -ti db psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

postgres=# CREATE TABLE tasks (id SERIAL PRIMARY KEY, description VARCHAR(100));
INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
SELECT * FROM tasks;
\q
CREATE TABLE
INSERT 0 2
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)

PS C:\Users\21315>
```

3.Stop and remove the container: *docker stop dbdocker rm db*

```
PS C:\Users\21315> docker stop db
db
PS C:\Users\21315> docker rm db
db
```

4.Start a new container with the same volume

docker run --name=new-db -d -v postgres_data:/var/lib/postgresql postgres:18

```
PS C:\Users\21315> docker run --name=new-db -d -v postgres_data:/var/lib/postgresql postgres:18
fc2ab5024f678d9eac9ba01e33e6d6ed2f7e2902043bcf5811341607bf5a049d
PS C:\Users\21315>
```

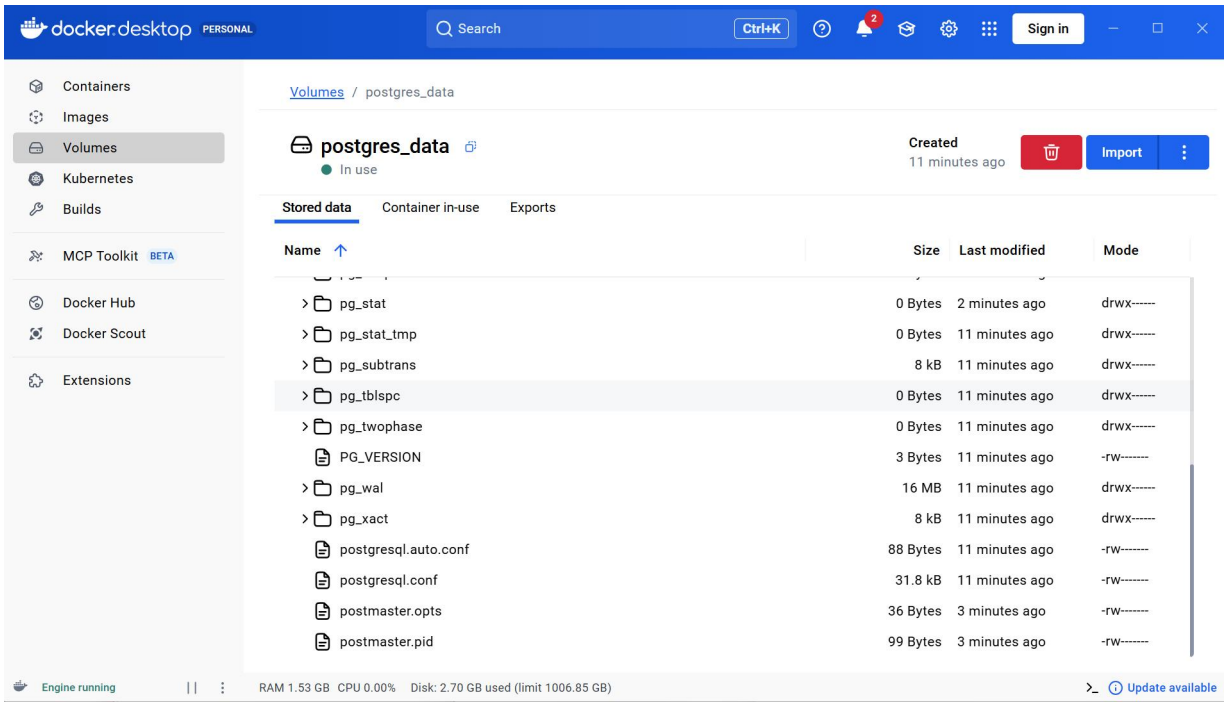
5.Verify data persistence

*docker exec -ti new-db psql -U postgres -c "SELECT * FROM tasks"*

```
PS C:\Users\21315> docker exec -ti new-db psql -U postgres -c "SELECT * FROM tasks"
 id | description
----+-------------
  1 | Finish work
  2 | Have fun
(2 rows)

PS C:\Users\21315>
```

6.View volume contents in Docker Desktop

Go to Volumes → postgres_data → Stored Data tab.



7.Clean up

docker rm -f new-dbdocker volume rm postgres_data

Experiment 2.4 – Sharing Local Files with Containers (Bind Mounts)

Objective:

Use a bind mount to serve a custom HTML file from the host into an Apache httpd container.

Experimental procedure:

1.Stop conflicting container (if any)

The welcome-to-docker container from Exp 1 was still using port 8080.

docker stop inspiring_liskov

Create project folder and HTML file

Create public_html folder on host (e.g., E:\cloud_lab\Lab4\public_html).

Inside it, create index.html with the provided whale ASCII art.

docker-compose.yml | Lab.2.txt | compose.yml | index.html

文件 编辑 查看

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title> My Website with a Whale & Docker!</title>
</head>
<body>
<h1>Whalecome!!</h1>
<p>Look! There's a friendly whale greeting you!</p>
<pre id="docker-art">
  ##        .
 ## ## ##       ==
 ## ## ## ## ##   ===
 /"""""""""""""""\   / ===
{                  / ===-
\       O          /
\   \        /
 \    \       /

Hello from Docker!
</pre>
</body>
</html>
```

行 23，列 8 | 445 个字符 | 纯文本 | 100% | Windows (CRLF) | UTF-8

2.Run httpd container with bind mount

cd E:\cloud_lab\Lab4\public_htmldocker run -d --name my_site -p 8080:80 -v ./:/usr/local/apache2/htdocs/ httpd:2.4

```
PS C:\Users\21315> docker run -d -p 8080:80 --name my_site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
1935f25c7dc4: Pull complete
5b16784cbecd: Pull complete
d32ec5ee95cb: Pull complete
4f4fb700ef54: Pull complete
b267b545226b: Pull complete
555e17187742: Download complete
2a2591e319bc: Download complete
Digest: sha256:89a0b59e6d7285f00fc8df952a1579c31eb035cbdccb96690051e8bb6432cbea
Status: Downloaded newer image for httpd:2.4
f01bf275616e306feae6a3d3a0d92155eb8bed3072361b1aa23ae5149ce3f31e
PS C:\Users\21315>
```

3.Verify container is running

docker ps | findstr my_site

```
PS E:\云计算导论\Lab2\Lab2.4\public_html> docker run -d --name my_site -p 8080:80 -v ./:/usr/local/apache2/htdocs/ httpd
:2.4
d4202ab369a9e9d636cbca9c3cc2f00da6402330efaa377a51bbf39ab551e2fb
PS E:\云计算导论\Lab2\Lab2.4\public_html>
```

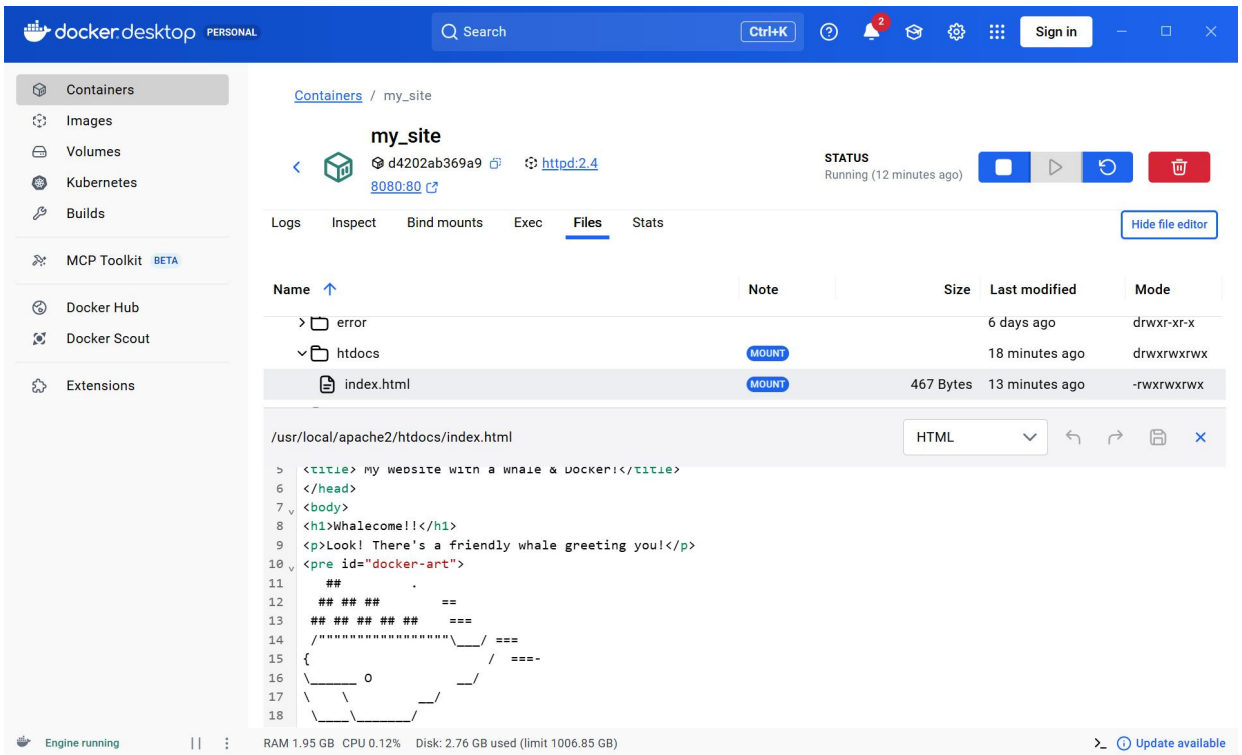
Open browser

Go to http://localhost:8080.

First attempt showed the old “Congratulations!!!” page (cached).

4.Check mounted files inside container (via Docker Desktop)

Containers → my_site → Files → /usr/local/apache2/htdocs/index.html.



Errors encountered

Port 8080 was already used by the old welcome-to-docker container.

Fix: Stopped that container with docker stop inspiring_liskov.

Browser still showed old page even after stopping the old container.

Cause: Browser cache.

Fix: Used private browsing mode (or clear cache). After that, the custom page appeared.

Experiment 2.5 – Multi- container Applications

Objective:

Deploy a counter web app (Nginx + Node.js + Redis) manually with docker run, then compare with Docker Compose.

Experimental procedure:

2.5.1 Preparation – Clone the sample project

cd E:\CloudLab\Lab5git clone https://github.com/dockersamples/nginx-node-redis .

```
PS E:\云计算导论\Lab2\Lab2.5> git clone https://github.com/dockersamples/nginx-node-redis .
Cloning into '.'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 82 (delta 22), reused 18 (delta 18), pack-reused 54 (from 1)
Receiving objects: 100% (82/82), 76.62 KiB | 69.00 KiB/s, done.
Resolving deltas: 100% (26/26), done.
PS E:\云计算导论\Lab2\Lab2.5> dir


    目录: E:\云计算导论\Lab2\Lab2.5


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          2026/4/28      9:29                nginx
d-----          2026/4/28      9:29                web
-a----          2026/4/28      9:29            378 compose.yml
-a----          2026/4/28      9:29            181 CONTRIBUTING.md
-a----          2026/4/28      9:29          11558 LICENSE
-a----          2026/4/28      9:29           3632 README.md


PS E:\云计算导论\Lab2\Lab2.5>
```

2.5.2 Manual deployment with docker run

1.Build images

cd nginxdocker build -t nginx .cd .

```
PS E:\云计算导论\Lab2\Lab2.5> cd nginx
PS E:\云计算导论\Lab2\Lab2.5\nginx> docker build -t nginx .
[+] Building 44.2s (8/8) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.1s
 => => transferring dockerfile: 141B                                                                                  0.0s
 => [internal] load metadata for docker.io/library/nginx:1.29                                                         4.8s
 => [internal] load .dockerignore                                                                                     0.0s
 => => transferring context: 2B                                                                                       0.0s
 => [1/3] FROM docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d19  42.5s
 => => resolve docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199  0.0s
 => => sha256:4677c2a9a3d4f9290cb784d95a9e16378655ecdd7df9e77668d3915262730d0b 956B / 956B                           1.4s
 => => sha256:85c66128325abc04138f6944d943e5279375665f6dbefe7f4f6b5e9646d31998 629B / 629B                           1.6s
 => => sha256:677c631968686eeb23ab8dd436d49bde041266df5d8952f03d7a8c418643d4b5 1.21kB / 1.21kB                       1.6s
 => => sha256:801a1ad15b4e00add388aca409568400fb8071019d6ba83995f43170af7656fe 1.40kB / 1.40kB                       0.8s
 => => sha256:ce776bbcda0d6bf4da8df324b82066a03f45bfbbbe520df535293ae069994e84 33.16MB / 33.16MB                    40.9s
 => => sha256:ff048f1f2159a060f69b1861ea262b839cc6e77a9389848929f70275eb7c9e29 403B / 403B                           0.8s
 => => extracting sha256:ce776bbcda0d6bf4da8df324b82066a03f45bfbbbe520df535293ae069994e84                            0.5s
 => => extracting sha256:85c66128325abc04138f6944d943e5279375665f6dbefe7f4f6b5e9646d31998                            0.0s
 => => extracting sha256:4677c2a9a3d4f9290cb784d95a9e16378655ecdd7df9e77668d3915262730d0b                            0.0s
 => => extracting sha256:ff048f1f2159a060f69b1861ea262b839cc6e77a9389848929f70275eb7c9e29                            0.0s
 => => extracting sha256:677c631968686eeb23ab8dd436d49bde041266df5d8952f03d7a8c418643d4b5                            0.0s
 => => extracting sha256:801a1ad15b4e00add388aca409568400fb8071019d6ba83995f43170af7656fe                            0.0s
 => [internal] load build context                                                                                     0.1s
 => => transferring context: 222B                                                                                     0.0s
 => [2/3] RUN rm /etc/nginx/conf.d/default.conf                                                                       0.4s
 => [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf                                                              0.0s
 => exporting to image                                                                                                0.3s
 => => exporting layers                                                                                               0.1s
 => => exporting manifest sha256:e59fdbeade9788671b6d2e50fceedf5304ead4c1575a93f8e2479dc153c19cd0                     0.0s
 => => exporting config sha256:775b8ce159797357987 19f556df1c1cb930f447e8285eb7dd2ca6136d463b481                      0.0s
 => => exporting attestation manifest sha256:68ad7dac0ace9209b6051b5f5996d5ea686677a4680f3ef62e9fbde1368e3790         0.0s
 => => exporting manifest list sha256:10d965c565d70ab1032f7a20cb338a9c4cbbdfd98ab1edea343d74ea8e9511fb                0.0s
 => => naming to docker.io/library/nginx:latest                                                                       0.0s
 => => unpacking to docker.io/library/nginx:latest                                                                    0.0s
PS E:\云计算导论\Lab2\Lab2.5\nginx>
```

webdocker build -t web .cd ..

```
PS E:\云计算导论\Lab2\Lab2.5\web> docker build -t web .
[+] Building 1340.4s (10/10) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.0s
 => => transferring dockerfile: 175B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/node:21                                                              1.9s
 => [internal] load .dockerignore                                                                                       0.0s
 => => transferring context: 2B                                                                                         0.0s
 => [1/5] FROM docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e3  1330.8s
 => => resolve docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d   0.0s
 => => sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88 449B / 449B                             1.1s
 => => sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150 1.25MB / 1.25MB                        10.0s
 => => sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb 49.72MB / 49.72MB                    1329.7s
 => => sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd 3.37kB / 3.37kB                         1.9s
 => => sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2 211.21MB / 211.21MB                    80.4s
 => => sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853796fafa221d850d42 24.05MB / 24.05MB                      31.0s
 => => sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3 64.14MB / 64.14MB                      56.6s
 => => sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7 49.58MB / 49.58MB                      22.9s
 => => extracting sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7                               0.9s
 => => extracting sha256:891494355808bdd3db5552f0d3723fd0fa826675f774853796fafa221d850d42                               0.3s
 => => extracting sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3                               1.1s
 => => extracting sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2                               2.9s
 => => extracting sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd                               0.0s
 => => extracting sha256:4684276225df21ffeac91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb                               0.9s
 => => extracting sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150                               0.0s
 => => extracting sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88                               0.0s
 => [internal] load build context                                                                                       0.1s
 => => transferring context: 17.30kB                                                                                    0.0s
 => [2/5] WORKDIR /usr/src/app                                                                                          0.1s
 => [3/5] COPY package.json package-lock.json ./                                                                        0.0s
 => [4/5] RUN npm ci                                                                                                    4.9s
 => [5/5] COPY ./server.js ./                                                                                           0.0s
 => exporting to image                                                                                                  0.5s
 => => exporting layers                                                                                                 5.3s
 => => exporting manifest sha256:a7323160ef8185f7a4b7d238e6ea80e014ccc53f639f5fd434538c1a0bc9305c                       0.0s
 => => exporting config sha256:4f8ac031ff856a26f5cf18f11ab3e61e8fcc28c1ea3b63787d7d5208616f5e46                         0.0s
 => => exporting attestation manifest sha256:6c570d0eda0b62f5af7ccf928e1967d0c229f12a1efa21a59cd8a71bb771bcde           0.0s
 => => exporting manifest list sha256:efac19c5a91cb88eec5925e3e56677ce18e71e3fed17cb4d33bf7618f78a80ef                  0.0s
 => => naming to docker.io/library/web:latest                                                                           0.0s
 => => unpacking to docker.io/library/web:latest                                                                        0.0s
PS E:\云计算导论\Lab2\Lab2.5\web> docker build -t web .docker build -t web .docker build -t web .docker build -t web .
```

2.Create network

docker network create sample-app

```
PS E:\云计算导论\Lab2\Lab2.5> docker network create sample-app
5cef0d499615a544ccbb169c7dd070bf1009a6d9efc346535410d0f96b7104db
PS E:\云计算导论\Lab2\Lab2.5>
```

3.Run containers

docker run -d --name redis --network sample-app --network-alias redis redisdocker

```
PS E:\云计算导论\Lab2\Lab2.5> docker run -d --name redis --network sample-app --network-alias redis redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
2b8951cd65b5: Pull complete
4f4fb700ef54: Pull complete
14d3b17b0010: Pull complete
d55df8e0f53d: Pull complete
9e0f42ff49e1: Pull complete
97afe6025aa9: Pull complete
f426aaaf8949: Download complete
ff0fc1b2ba98: Download complete
Digest: sha256:832d7785830f3f4b559300e6191fc914b15642c1935252338825cf4332200148
Status: Downloaded newer image for redis:latest
a15e45ad98accdf1f3bb7f679516306fb0e77fa502432d129a4994fe45f3e932
PS E:\云计算导论\Lab2\Lab2.5>
```

run -d --name web1 -h web1 --network sample-app --network-alias web1 webdocker

run -d --name web2 -h web2 --network sample-app --network-alias web2 webdocker

```
PS E:\云计算导论\Lab2\Lab2.5> docker run -d --name web1 -h web1 --network sample-app --network-alias web1 web
70b188a3077b6ecf3075a9e301cecde534cb492f58be6d060f310ffc0c9a0e90
PS E:\云计算导论\Lab2\Lab2.5> docker run -d --name web2 -h web2 --network sample-app --network-alias web2 web
b647c900feda4a2cad9f4c790d96119a3883c03a47d0ccdbc9a559cad9421ae8
PS E:\云计算导论\Lab2\Lab2.5>
```

run -d --name nginx --network sample-app -p 80:80 nginx

```
PS E:\云计算导论\Lab2\Lab2.5> netstat -ano | findstr :80
  TCP    10.0.175.131:54416     180.110.193.149:8080   ESTABLISHED     19668
PS E:\云计算导论\Lab2\Lab2.5> docker run -d --name nginx --network sample-app -p 80:80 nginx
4e12a60b7fa0af334f6afee751d6f29bf69156546549727895 7e5efa1ed2d21d
PS E:\云计算导论\Lab2\Lab2.5>
```

4.Check running containers

docker ps

```
PS E:\云计算导论\Lab2\Lab2.5> docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS          PORTS
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
NAMES
4e12a60b7fa0   nginx               "/docker-entrypoint.…"   25 seconds ago   Up 21 seconds   0.0.0.0:80->80/tcp, [::]:8
0->80/tcp
nginx
b647c900feda   web                 "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes
web2
70b188a3077b   web                 "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes
web1
a15e45ad98ac   redis               "docker-entrypoint.s…"   4 minutes ago    Up 4 minutes    6379/tcp
redis
f22879f67d58   gitea/gitea:latest  "/usr/bin/entrypoint…"   45 hours ago     Up 3 hours      0.0.0.0:3000->3000/tcp, [:
:]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp   gitea
PS E:\云计算导论\Lab2\Lab2.5>
```

5.Test in browser

Open http://localhost (port 80).

Refresh several times.

2.5.3 Use Docker Compose

1.Verify compose.yml exists：

cat compose.yml

```
PS E:\云计算导论\Lab2\Lab2.5> cat compose.yml

services:
  redis:
    image: redis
    ports:
      - '6379:6379'
  web1:
    restart: on-failure
    build: ./web
    hostname: web1
    ports:
      - '81:5000'
  web2:
    restart: on-failure
    build: ./web
    hostname: web2
    ports:
      - '82:5000'
  nginx:
    build: ./nginx
    ports:
    - '80:80'
    depends_on:
    - web1
    - web2
```

2.Start with Compose: *docker compose up -d --build*

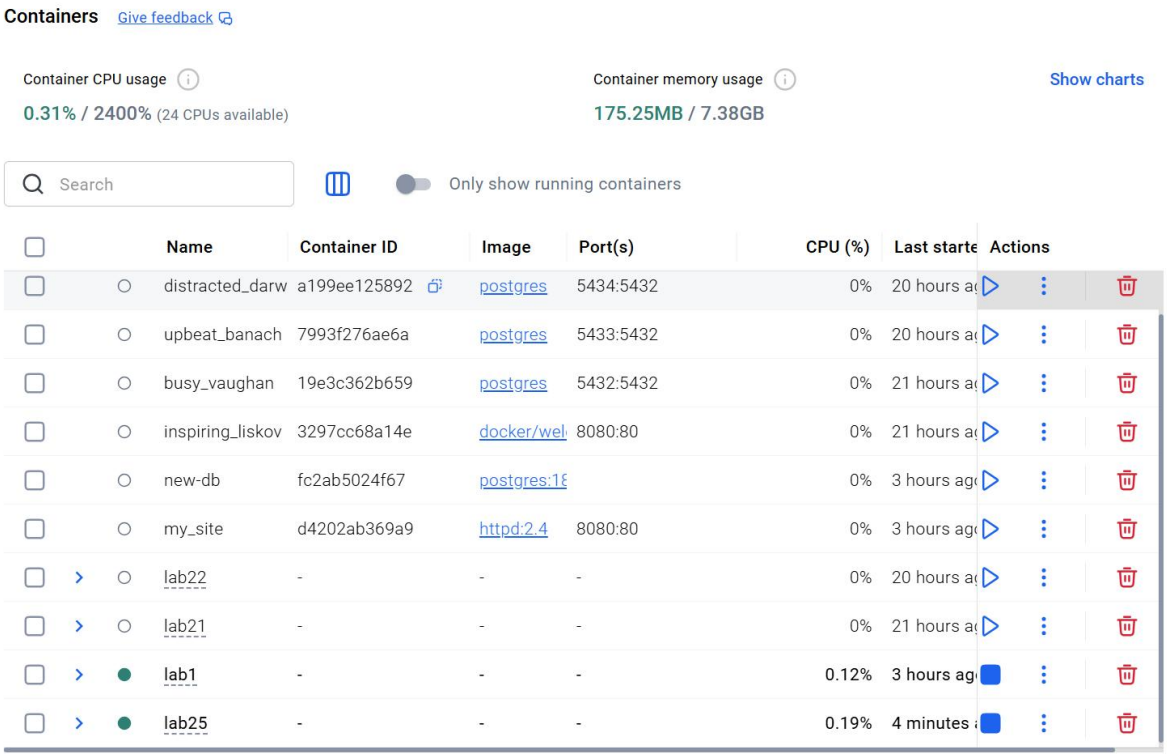
```
PS E:\云计算导论\Lab2\Lab2.5> docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.39kB done
#1 DONE 0.0s

#2 [web2 internal] load build definition from Dockerfile
#2 transferring dockerfile: 175B done
#2 DONE 0.0s

#3 [nginx internal] load build definition from Dockerfile
#3 transferring dockerfile: 141B 0.0s done
#3 DONE 0.0s

#4 [web2 internal] load metadata for docker.io/library/node:21
#4 ...

#5 [nginx internal] load metadata for docker.io/library/nginx:1.29
#5 DONE 1.3s
```

```
[+] up 8/8
 ✔Image lab25-web2        Built
 ✔Image lab25-nginx       Built
 ✔Image lab25-web1        Built
 ✔Network lab25_default   Created
 ✔Container lab25-redis-1 Started
 ✔Container lab25-web1-1  Started
 ✔Container lab25-web2-1  Started
 ✔Container lab25-nginx-1 Started
PS E:\云计算导论\Lab2\Lab2.5>
```

3. Test again – browser http://localhost works identically.

View in Docker Desktop

Containers now have names like lab5-web-1, lab5-nginx-1, etc.



Conclusion

All five experiments were successfully completed. I learned:

How to publish container ports and access services from the host.How to override default settings (multiple instances, custom networks, resource limits, command overrides).

How to persist data using Docker volumes.How to share live files between host and container via bind mounts.How to manage multi- container applications both manually and with Docker Compose, and why Compose simplifies the process.